

DIGITAL FORENSICS LABORATORY · LAB 07 · SERIES FINALE

Memory Forensics

Working paper — RAM acquisition, Volatility plugins, and the four hiding places

Name	Student ID (NIM)	Class / Group
------	------------------	---------------

Date	Instructor / TA	Session grade
------	-----------------	---------------

1. Lab Overview

This working paper accompanies Lab 07, the final lab of the series. A memory dump is a snapshot of a machine while it was alive: processes and their parentage, live network connections, the clipboard, typed commands, environment blocks - evidence that dies at power-off and never touches the disk. You will acquire memory with DumpIt, orient inside a dump with imageinfo, work the core Volatility plugins (pslist, pstree, netscan, cmdline, envvars, clipboard, consoles, cmdscan), and close the series with the four hiding places of the upstream ransomware-gang case. Complete the pre-lab before the session, cite plugin + PID in every answer, and close with a reflection on the whole series.

1.1 Learning objectives

- Explain what volatile memory preserves that disks do not, and why acquisition order matters.
- Acquire a memory dump with DumpIt (and name the alternatives: FTK Imager, Redline, hiberfil.sys).
- Determine a dump's profile with imageinfo and explain why the profile is required.
- Reconstruct process genealogy with pslist/pstree and spot anomalies by name, parent, and start time.
- Map live network connections with netscan and join them to owning processes by PID.
- Extract how processes were launched (cmdline) and their environment blocks (envvars).
- Hunt human-layer secrets: clipboard contents, console history (consoles/cmdscan), browser trails.
- Sweep dumps with strings in both ASCII and UTF-16LE as the safety net beneath the plugins.

1.2 Environment and files

Environment: Kali Linux with Volatility 2 (python2) installed per the lab README, or Volatility 3 with translated commands. Challenge A's dump is downloaded from the link in the README (it is large - start the download early). The two course artifacts are small and live in the repo.

Table 1.1 — Lab files

File	Purpose
Google Drive dump (see README)	Challenge A — the leader's Windows 7 memory dump (upstream)
files/triage_output.txt	Challenge B (new) — a Volatility export from a real imaging-fault case
files/mem_strings.bin	Challenge C (new) — a raw strings-hunt blob with layered artifacts

1.3 What to submit

- Section 1.4 pre-lab, answered before the session (worth 10 points).
- This working paper as PDF, named NIM_Name_Lab07.pdf.
- Your evidence folder NIM_Lab07/evidence/ containing every numbered exhibit (B-01, B-02, and so on).
- Answers that cite the plugin + PID (or strings mode + offset) - not vague descriptions.

1.4 Pre-Lab Check

Answer these before the session, using the lab README and Section 2 of this paper as references. One or two sentences per question are enough - precision beats length.

P1. Name three things a memory dump can contain that a disk image cannot, and explain why power-off destroys them.

Answer:

Two sentences maximum.

P2. What is a Volatility profile, what does imageinfo do, and what happens if you analyze with the wrong profile?

Answer:

Two sentences maximum.

P3. In pslist/pstree output, which three columns together flag a suspicious process even before you read its name?

Answer:

Two sentences maximum.

P4. What is the difference between cmdscan and consoles, and what extra evidence does consoles provide?

Answer:

Two sentences maximum.

P5. Windows stores most in-memory text as UTF-16LE. Which strings flag reads that encoding, and why does plain strings miss it?

Answer:

Two sentences maximum.

2. Theory Primer

2.1 Volatile memory as evidence

RAM holds the running state: process objects and their genealogy, loaded code (including fileless payloads), live sockets, decrypted secrets, clipboard data, and console buffers. None of it exists as a file - power-off or even heavy paging destroys or dilutes it. That is why incident response acquires memory FIRST, and why fileless malware made memory forensics a core discipline rather than a specialty.

2.2 Volatility and the profile

Volatility is an open-source framework (version 2 in this lab, Python 2; version 3 is the modern rewrite with auto-detection) that parses a dump's kernel structures. Because RAM has no filesystem, the tool needs the memory map of the exact OS build: the profile (e.g. Win7SP1x64). imageinfo suggests profiles - take the first unless output looks wrong. Every subsequent command carries --profile <name>.

2.3 The core plugin set

Table 2.1 — Plugins you will actually use

Plugin	Answers	Read it for
pslist / pstree	which processes were alive	names, PID/PPID, threads, handles, start times, genealogy
cmdline	how each was launched	full command line, image path, encoded arguments
netscan	what was talking to whom	protocol, local/foreign addresses, state, owning PID
envvars	what each process believed	environment blocks - odd variables stand out
clipboard	what the user copied	CF_TEXT and other formats, still resident
consoles / cmdscan	what was typed	prompt-by-prompt history (consoles) vs commands only (cmdscan)
iehistory	where the browser went	URLs, including search queries

2.4 Strings in two encodings

Windows text in memory is usually UTF-16LE (two bytes per character, second byte zero for ASCII) - plain strings misses it. Sweep with `strings -el` for wide text and `plain strings` for ASCII; both are cheap, so run both. Strings is the safety net when plugins fail: carved regions, damaged structures, unknown profiles. It is also where decoys live - decode every layer (base64, ROT13, URL encoding) before believing a flag.

2.5 The correlation loop

No single plugin convicts. The method: `imageinfo` for the profile; `pslist/pstree` for the odd process; `cmdline` for how it launched and where it lives; `netscan` for its connections (join by PID); `consoles/cmdscan/envvars/clipboard` for the human layer; `strings` to verify. Every finding is reported as plugin + PID - the citation format of this lab.

3. Guided Exercises

Drills 3.A-3.B need the Challenge A dump (download early!); 3.C-3.D work on the course artifacts. Every output is an exhibit: capture the plugin table, number it (B-01, B-02, and so on), and log it in Appendix B.

3.A Orient in the dump

- **1.** Record the dump's MD5 and size once the download completes.
- **2.** Run `imageinfo`; note the suggested profiles, number of processors, and image date/time.
- **3.** Pick the first suggested profile; run `pslist` and record the total process count.

- **4.** From the pslist output, record start time and PID of explorer.exe.

3.B Walk the plugins

- **1.** pstree: draw (on paper) the subtree under explorer.exe.
- **2.** cmdline: record the full command line of any browser process.
- **3.** netscan: list every ESTABLISHED connection with its owning PID and process name.
- **4.** envvars: grep the output for flag/token/secret/key and record any hits with their PIDs.
- **5.** clipboard and consoles: screenshot both outputs in full.

3.C Triage export warm-up (files/triage_output.txt)

- **1.** Record the file's MD5. What five plugin sections does it contain?
- **2.** From section [1]: which processes were started AFTER 09:00? List PID, name, and start time.
- **3.** From section [2]: quote the suspicious subtree (two lines) exactly as printed.
- **4.** From section [4]: list every connection owned by PID 3472 or 3596.

3.D Strings warm-up (files/mem_strings.bin)

- **1.** Record the file's MD5 and size.
- **2.** Run plain strings and count how many URL lines appear; quote one.
- **3.** Run strings -el and quote one environment-looking line.
- **4.** Decode the 'backup blob' base64 line and write the result.

4. Challenges

Challenge A is the upstream ransomware-gang case - solved together in the Deep Dive session (its dump is the external download; the flags are in flag{xxxx} format). Challenges B and C are yours: work on copies, hash before and after, cite plugin + PID or strings mode. B and C use ctf{xxxx}.

4.1 Challenge A — the leader's dump (solved in the Deep Dive)

Four flags, four hiding places: something was copied to the clipboard, searched on the internet, saved in an environment variable, and executed as a command. The Deep Dive demonstrates the exact command per hiding place - your deliverable is the four flags backed by YOUR plugin screenshots, each cited as plugin + output line.

4.2 Challenge B — the triage export (files/triage_output.txt)

- **1.** Hash the file. What case, tool, profile, and acquisition time does the header declare? What misfortune produced this export?

- **2.** From pslist: which process has an anomalous name? Record its PID, PPID, parent's name, and start time.
- **3.** From pstree: quote the two-line subtree that makes this process suspicious.
- **4.** From cmdline: where does the suspicious binary live, and what did it spawn? Decode the child's -enc argument fully and write the decoded command.
- **5.** From netscan: list both foreign endpoints this incident talked to, with ports, states, and owning PIDs.
- **6.** From consoles: what did the user type last, and what does it reveal? Write the handover token exactly.
- **7.** In two sentences: why does no single section prove the incident, and which three sections together do?

Hints: (1) The odd one out is NOT the process with the strangest-sounding name of the whole list - check parents and start times first. (2) -enc means base64 of UTF-16LE; CyberChef or python decode both layers. (3) The console section ends with the case's punchline.

Flag B: ctf{_____}

4.3 Challenge C — the strings hunt (files/mem_strings.bin)

- **1.** Hash the file. Run plain strings: list the four artifact TYPES you can find (clipboard-like text, URL, note, blob) and quote one line of each.
- **2.** Run strings -el: quote every flag-LIKE string it reveals, in full.
- **3.** One 'flag' is an impostor planted in an environment variable. Which is it, and what marks it as a decoy?
- **4.** The 'note-to-self' line contains a pgs{...} string. What is it, and what transformation reveals its true form? Why is it still not the answer?
- **5.** Decode the base64 'backup blob' and write the result.
- **6.** The real flag lives in exactly one artifact type. Which one, in which encoding, and what is the token?
- **7.** In two sentences: why would plain strings have failed an examiner who never used -el, and which Lab 01 lesson does this repeat?

Hints: (1) Windows keeps its text wide - check both encodings before concluding anything. (2) pgs is what ctf becomes under ROT13. (3) certutil lines in console logs are a classic malware staging pattern - read what follows them.

Flag C: ctf{_____}

5. Analysis Questions

Questions 1-3 condense the upstream case; questions 4-12 extend it and close the series. Cite plugin + PID (or strings mode) in every answer.

1. The leader's dump (Challenge A):

- a. Which plugin answers each of the four hiding places? Name all four commands.
- b. Why is imageinfo the mandatory first command for this dump?
- c. What do you cite, precisely, when reporting each found flag?

Answer:

Write your complete answer, citing the plugin, PID, or strings mode used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

2. Challenge B outcomes:

- a. Which process is the intrusion, and what three attributes made it stand out?
- b. What URL did the decoded -enc command contact, and via which PID's socket?

Answer:

Write your complete answer, citing the plugin, PID, or strings mode used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

3. Challenge C outcomes:

- a. Why did the real flag never appear in plain strings output?
- b. Name each decoy layer you decoded before reaching the true flag.

Answer:

Write your complete answer, citing the plugin, PID, or strings mode used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

4. Explain the difference between what pslist shows and what psscan can find. Which hiding technique does that difference expose?

Answer:

Write your complete answer, citing the plugin, PID, or strings mode used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

5. The incident-response rule is 'memory first, disk second, logs last'. Justify the order using what each layer loses over time.

Answer:

Write your complete answer, citing the plugin, PID, or strings mode used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

6. A colleague dumps memory with the machine still connected to the network. Name two risks this creates for evidence integrity.

Answer:

Write your complete answer, citing the plugin, PID, or strings mode used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

7. Why does -enc in a command line deserve automatic suspicion? Describe the full decoding chain from the blob to the readable command.

Answer:

Write your complete answer, citing the plugin, PID, or strings mode used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

8. The clipboard plugin shows the LAST copied item only. What does that limitation mean for hunting, and what other artifact might preserve older copies?

Answer:

Write your complete answer, citing the plugin, PID, or strings mode used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

9. Connect this lab to the series: name one artifact from each of Labs 4-6 that would corroborate the 45.9.148.23 C2 you found in memory today.

Answer:

Write your complete answer, citing the plugin, PID, or strings mode used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

10. Why must the memory dump be hashed at receipt and after analysis? What does a mismatch prove - and what does it not?

Answer:

Write your complete answer, citing the plugin, PID, or strings mode used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

11. Volatility 3 auto-detects profiles while version 2 asks you to choose. Give one scenario where auto-detection is a risk rather than a convenience.

Answer:

Write your complete answer, citing the plugin, PID, or strings mode used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

12. Final reflection prompt: management asks 'what did the seven labs teach us about this intrusion?'. Draft the three-sentence answer, naming one artifact per sentence.

Answer:

Write your complete answer, citing the plugin, PID, or strings mode used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

6. Reflection and Self-Assessment

Summarize your findings from this lab - and, since this is the series finale, one sentence on what each of the seven labs contributed to your investigation skill set.

Reflection:

This lab's findings + one line per lab of the series.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

Table 6.1 — Self-assessment checklist

I can	Yes	Not yet
Explain why memory evidence dies at power-off		
Determine and apply a Volatility profile		
Walk pslist/pstree/cmdline/netscan and correlate by PID		
Hunt the human layer: clipboard, consoles, envvars		
Sweep dumps with strings in both encodings		
Decode layered payloads (base64, ROT13, URL, UTF-16)		
Report findings as plugin + PID citations		

7. Appendices

Appendix A — Tool cheat sheet

Task	Command / action
Acquire memory	DumpIt.exe (y to confirm) - or FTK Imager / Redline
Identify profile	python2 vol.py -f dump.mem imageinfo
Process list / tree	vol.py ... pslist pstree
Command lines	vol.py ... cmdline [-p PID]
Network	vol.py ... netscan
Environment	vol.py ... envvars grep -iE 'flag token'
Clipboard	vol.py ... clipboard
Console history	vol.py ... consoles (commands only: cmdscan)
Browser trail	vol.py ... iehistory grep -i search
Hidden processes	vol.py ... psscan (compare against pslist)
Strings both ways	strings dump.mem less strings -el dump.mem less
Decode -enc blob	CyberChef: FromBase64 + UTF-16LE decode
ROT13 / URL decode	CyberChef ROT13; python urllib unquote_plus

Appendix B — Evidence log

One row per exhibit. Keep numbering continuous across the whole session.

What good evidence looks like — model exhibit:

```
[B-04] 2026-11-03 11:20 Volatility netscan
cmd: vol.py ... netscan | findstr 3472
      192.168.0.52:49202 -> 45.9.148.23:8080 ivkui.exe
-> captured as evidence/B-04_netscan_ivkui.png
```

- The plugin, the filter, and the decisive line are fully visible.
- The PID and owner tie the finding to a process, not a vibe.
- One exhibit supports one finding.
- Bad evidence: 'netscan looked suspicious' with no PID or line quoted.

No.	Time	Activity / Plugin	Source file	Notes
B-01				
B-02				
B-03				
B-04				

No.	Time	Activity / Plugin	Source file	Notes
B-05				
B-06				
B-07				
B-08				
B-09				
B-10				

Appendix C — Hash verification form

Fill one row per artifact (the dump, the export, the blob): hash when received (start) and after analysis (end). Identical values prove faithful handling.

No	Artifact	Algorithm	Hash (start)	Hash (end)	Match?
1					Yes / No
2					Yes / No
3					Yes / No
4					Yes / No

Appendix D — Glossary

Term	Meaning in this lab
memory dump	a raw capture of RAM: .raw, .mem, .vmem, .bin extensions
volatile evidence	data that vanishes at power-off: processes, sockets, clipboard
Dumplt / Redline	one-click Windows dumper / FireEye's acquisition agent
hiberfil.sys	compressed RAM image from the last boot - the offline fallback
profile	the OS/arch/SP memory map Volatility 2 needs to parse a dump
imageinfo	plugin that suggests the profile and reports image times
PID / PPID	process id / parent process id - the genealogy columns
cmdline	plugin: full command line per process
netscan	plugin: sockets and connections with owning PIDs
envvars	plugin: per-process environment blocks
clipboard / consoles / cmdscan	what was copied / typed-with-output / typed

Term	Meaning in this lab
iehistory	plugin: browser URLs incl. search queries
-enc	PowerShell encoded-command switch: base64 of UTF-16LE
UTF-16LE (wide)	two-byte Windows text; strings -el extracts it
ROT13	letter rotation; 'ctf' becomes 'pgs'
fileless malware	malware that lives only in RAM - memory is its only crime scene
plugin + PID citation	this lab's evidence format for every claim

Honesty statement: by submitting this working paper I declare that all answers and evidence are my own work, that original evidence files were never modified, and that I followed the lab's ethics rules.

Student signature: _____ Date: _____

Instructor / TA signature: _____ Grade: _____